

```

# EJERCICIO 1: Mezcla de fertilizantes
# =====
# Función Objetivo (a maximizar):  $Z = 180 \cdot X_1 + 220 \cdot X_2$ 
# scipy.optimize.linprog minimiza por defecto, así que invertimos los signos de la función objetivo.
c = [-180, -220]

# Coeficientes de las restricciones (lado izquierdo de la inecuación)
A = [
    [3, 4], # Restricción Compuesto A
    [2, 1], # Restricción Compuesto B
    [5, 3]  # Restricción Compuesto C
]

# Valores máximos de las restricciones disponibles (lado derecho)
b = [120, 80, 150]

# Límites de no negatividad para  $X_1$  y  $X_2$  ( $\geq 0$ )
limites = [(0, None), (0, None)]

# Resolución mediante Programación Lineal
res_ej1 = linprog(c, A_ub=A, b_ub=b, bounds=limites)

print("--- EJERCICIO 1 ---")
if res_ej1.success:
    print(f"X1 (Fertilizante F1): {res_ej1.x[0]:.2f} ton")
    print(f"X2 (Fertilizante F2): {res_ej1.x[1]:.2f} ton")
    print(f"Ganancia máxima (Z): ${-res_ej1.fun:.2f}")

```

```

... --- EJERCICIO 1 ---
X1 (Fertilizante F1): 21.82 ton
X2 (Fertilizante F2): 13.64 ton
Ganancia máxima (Z): $6927.27

```

```

# =====
# EJERCICIO 2: Producción semanal
# =====
print("\n--- EJERCICIO 2 ---")
# Condiciones iniciales
S0 = 200
M0 = 80

# Iteración para calcular semanas 1 y 2
S1 = 0.6 * S0 + 0.2 * M0 + 40
M1 = 0.1 * S0 + 0.5 * M0 + 20

S2 = 0.6 * S1 + 0.2 * M1 + 40
M2 = 0.1 * S1 + 0.5 * M1 + 20

print(f"Semana 1: Sillas={S1:.1f}, Mesas={M1:.1f}")
print(f"Semana 2: Sillas={S2:.1f}, Mesas={M2:.1f}")

# Cálculo de los puntos de equilibrio
# El equilibrio es cuando  $S_{\{n+1\}} = S_n$  y  $M_{\{n+1\}} = M_n$ 
#  $S = 0.6S + 0.2M + 40 \rightarrow 0.4S - 0.2M = 40$ 
#  $M = 0.1S + 0.5M + 20 \rightarrow -0.1S + 0.5M = 20$ 
A_eq2 = np.array([[0.4, -0.2], [-0.1, 0.5]])
b_eq2 = np.array([40, 20])

# Resolviendo el sistema de ecuaciones lineales
eq_prod = np.linalg.solve(A_eq2, b_eq2)
print(f"Equilibrio: Sillas={eq_prod[0]:.2f}, Mesas={eq_prod[1]:.2f}")

```

```

--- EJERCICIO 2 ---
Semana 1: Sillas=176.0, Mesas=80.0
Semana 2: Sillas=161.6, Mesas=77.6
Equilibrio: Sillas=133.33, Mesas=66.67

```

```

# =====
# EJERCICIO 3: Epidemia en una ciudad
# =====
print("\n--- EJERCICIO 3 ---")
beta = 0.00003
gamma = 0.1

# La derivada dI/dt = beta*(10000 - I)*I - gamma*I se iguala a 0 para el equilibrio.
# Equilibrio 1 (trivial): Extraer factor común I. Si I = 0, todo es 0.
eq_I_1 = 0

# Equilibrio 2 (endémico): Lo que queda dentro del paréntesis se iguala a 0.
# beta*(10000 - I) - gamma = 0
eq_I_2 = (10000 * beta - gamma) / beta

print(f"Equilibrio 1 (Trivial): I = {eq_I_1} personas")
print(f"Equilibrio 2 (Endémico): I = {eq_I_2:.2f} personas")

```

```

--- EJERCICIO 3 ---
Equilibrio 1 (Trivial): I = 0 personas
Equilibrio 2 (Endémico): I = 6666.67 personas

```

```

# =====
# EJERCICIO 4: Modelo logístico de pesca
# =====
print("\n--- EJERCICIO 4 ---")
# Asumiendo r=0.4 (tasa de crecimiento estándar en lugar de 8.4)
r = 0.4
K = 1000
H = 80
P0 = 400

# Iteración poblacional años 1 y 2
P1 = P0 + r * P0 * (1 - P0 / K) - H
P2 = P1 + r * P1 * (1 - P1 / K) - H

print(f"Población Año 1 (P1): {P1:.1f} peces")
print(f"Población Año 2 (P2): {P2:.2f} peces")

# Cálculo de puntos de equilibrio en sistema no lineal discreto
# Igualamos P_{n+1} = P_n, lo que cancela las P a ambos lados dejando: r*P*(1 - P/K) - H = 0
def ecuacion_equilibrio(P):
    return r * P * (1 - P / K) - H

# Usamos fsolve de scipy para encontrar las raíces de la ecuación no lineal.
# Entregamos dos estimaciones iniciales (una baja y una alta) para capturar ambas raíces de la parábola.
puntos_equilibrio = fsolve(ecuacion_equilibrio, [200, 800])

print(f"Equilibrio 1 (Inestable / Umbral): {puntos_equilibrio[0]:.2f} peces")
print(f"Equilibrio 2 (Estable / Capacidad): {puntos_equilibrio[1]:.2f} peces")

```

--- EJERCICIO 4 ---

Población Año 1 (P1): 416.0 peces

Población Año 2 (P2): 433.18 peces

Equilibrio 1 (Inestable / Umbral): 276.39 peces

Equilibrio 2 (Estable / Capacidad): 723.61 peces